

Repository

<https://github.com/bluedragon2513/583-project>

Note: Repository is currently public. If any issues arise with access, please contact Anthony at (507) 513 - 3708, and he will get it public within minutes. A copy of the repository will be sent in the email anyhow.

Overview

Our project is an implementation similar to IBM's Watson Question Answering (QA) system. Given a category and a clue, our model attempts to return the title of the correct Wikipedia page.

The following data used for our project was provided under the D2L project folder:

- 100 questions from previous Jeopardy games, whose answers appear as Wikipedia pages. Questions are listed in a single file with 4 lines per question.
- A collection of approximately 280,000 Wikipedia pages, which include the correct answers for the above 100 questions. Pages are stored in 80 files (with each file containing several thousand pages).

Core Implementation

Any Information Retrieval (IR) system must create an index on some data and be able to search through that data. In this project, there are two core Python files that interface with Lucene via PyLucene: `index_wiki.py` and `search_jeopardy_core.py`.

For indexing, we retrieved document titles¹, document text, leading document text, and any categories found. The latter two are used in later parts of the implementation. Every dumped Wikipedia page is indexed as its own Lucene document. A simple regex is used to find document boundaries, amongst others to find templates (e.g. infoboxes), sections, redirects, categories, and references. In fact, most of the index is composed of non-template, non-reference, non-category, non-section header text to maintain textual purity (i.e. normalization). In addition, categories are extracted into their own field (but are not used for this implementation). The index is just Lucene's default inverted index, however we chose specific fields such as using the titles in both `StringField` and `TextField` with additional fields for the leads and categories. Tokenization, lowercasing, possessive stripping, stop-word elimination, and Porter stemming were done through Lucene's `EnglishAnalyzer`, but no lemmatization was done.

We actually tried to implement some more lemmatization using NLTK's²³ `WordNetLemmatizer` and Part-of-Speech tagger. Surprisingly, both were very slow and had a negative impact on our results. We saw a reduction in our $P@1$, and the runtime was about 8-10x slower (i.e. 4-5 minutes vs. 30 minutes). We hypothesize that the corpus is too small that certain words should preserve the intended spelling and word form in a hyper-edited encyclopedia. This is an atypical divergence from IR systems like search

¹ We indexed titles twice in the event that one use case has a side effect.

² NLTK = Natural Language Tool Kit: a library tused by many NLP researchers.

³ NLP = Natural Language Processing

engines which typically want an amorphous cloud of answers; our system is an IR system that wants a precise answer—and one specific to a category.

For querying, the entire clue verbatim is used to search against the content and title fields. The category and lead paragraph fields were not used at query time. Our initial reasoning behind this was that filtering out more words from the clue would leave out potential exact words that can distinguish correct articles versus near-misses. The terms were already preprocessed during indexing.

For searching, we use BM25. We attempted to match terms within the query with words in document titles and content. It is not an exact match, but weighs as many matched tokens as possible. For BM25, however, it has two parameters for controlling term frequency: normalization (k_1) and how much document length normalizes term frequency values (b)⁴. First, we chose to not have document length normalize term frequency values ($b = 0.0$) because Wikipedia articles are edited for conciseness, or succinctness as they themselves describe it⁵. This is only because the corpus consists of just Wikipedia articles. Any other corpus will probably fail using this IR system because that corpus will not have similar principles of conciseness.

Using this, we scored the algorithm and got a P@1 of 19% and MRR of 0.3150. We will later compare these results with our improved implementation.

Coding with LLMs

We used Claude Sonnet 4.6 to “vibe code” more than 99% of the code. No additional “skills” or context beyond that of this project were given to the LLM.

Before coding, we implemented a plan for what we were looking for from the LLM. We created the `prompting_document.txt` file that lists all the related files, gitignorable files, and a short list of important instructions. The LLM was very helpful when debugging/finding specific parts of the code that we wanted to improve upon later or tweak for proper implementation purposes.

Our major setback when prompting the LLM was compiling Lucene’s Java library due to a lot of dependencies needed that were not previously installed. Occasionally, the LLM would repeatedly fail when attempting to activate a partially setup Python virtual environment (i.e. no `venv/bin/activate` script was written).

Despite these setbacks, the LLM implemented about 90% of the way to our core system, upon which we fixed. The LLM was good at coding, but we understood that, as programmers, we must take responsibility for the code. Therefore, we read through every line of code to understand what was happening at various points in the code. We had, at the very least, a high level of abstract comprehension.

Many of the core ideas and considerations were made by us. For example, giving higher weights to textual precedence, assigning named entities more importance, and reasoning through how some parameters can be tweaked in BM25 (see footnote 4) to better improve scores. This requires actually looking through the documentation.

⁴ [Lucene Documentation](#)

⁵ [Wikipedia: Manual of Style](#)

We also tried to ask Claude to do an error analysis, but it gave some proper answers while others were blatantly in the wrong class. We gave these as an example in `error_class_*.txt` in the repository, and we wrote our own file `error_class_0_ours.txt`.

Measuring Performance

There are a few metrics for measuring the performance of an information retrieval system: Mean Average Precision (MAP), Mean Reciprocal Rank (MRR), and Precision at 1 ($P@1$), with the latter being our primary scoring measure.

The $P@1$ metric best measures the performance of our system because, for a project like Watson where there is a single shot at getting the correct answer, mirroring the real life equivalence of getting only one chance at answering. At first, we focused on measuring performance using $P@1$: from 10% (for the first three iterations) to 13%, then to 14%, and lastly to 19%.

The $P@1$ as our primary metric does not mean that the other metrics were not useful towards the analysis of the system's performance while we were creating the improved implementation. For example, we frequently used the Mean Reciprocal Rank to see if the system as a whole approached more correct answers, or if they were within grasp. These were helpful in measuring performance gains in many ideas such as synonym/redirects, Reciprocal Rank Fusion (RRF) for multiple searches, syntactic dependency indexing, dense indexing with retrieval for FAISS, and reranking with LLMs. Rerankings, for example, did not improve the $P@1$ because the MRR was not increasing due to any other improvements that could push correct answers from outside of the rankings to within.

Error Analysis

Our system correctly answers 19 questions, with the remaining 81 incorrectly answered. A lot of these questions can be answered because these jeopardy questions are often asking for a noun as an answer, which often is defined in encyclopedias like Wikipedia. Often, the clue matches a description, which is found in the articles.

For quite a few error classes, we found that it would be easy to code in some small function that can easily extract the answer if it matched a template query. However, this makes the IR overfitted to the dataset. Whenever there were new questions, the IR most likely could not retrieve answers correctly for those. Therefore, we sided with the fact that there should be no “hard coding” or query pattern matching. We refer to error classes as groups of errors that have similarities in failure, but those categories refer to the difficulty of the resulting solutions.

One error class has an indirect mapping and may require another step of knowledge to get to the answer. For instance, wordplay has some questions about “TIN” men and “COMPLETE DOMINATION”. These require the IR to understand that they are not searching for “TIN” or “DOMINATION”, but rather words containing those substrings. This falls into the solvable-using-hard-coding category.

Another error class is a categorical mismatch. When the category is “STATE OF THE ART MUSEUM” or “CAPITAL CITY CHURCHES”, these are referring to specific entity types like states and churches.

This also goes for “NAME THE PARENT COMPANY”. In these, BM25 is more likely to return words that are more similar to the query than the required entity type, which also falls into the solvable-using-hard-coding category like the previously mentioned error class.

A third error class worth mentioning were the clues/categories that provide too little information. For some queries surrounding 1980s songs that are hits like Michael Jackson’s “Beat It”, the word “beat” had a much higher score than “Michael Jackson” because of the matching word. This also overlaps with the another-step-of-knowledge error class, which can not be solved with a traditional IR. Likely, semantic information retrieval requires an LLM.

The last error class is having correct domains, but wrong entities were retrieved. For example, “The mast from the USS Maine is part of the memorial to the ship & crew at this national cemetery” was predicted to be the “USS Arizona Memorial”. It has the wrong answer despite being of the correct type. A categorical fix here is just parameter tweaking of BM25.

A more final error class was just having the wrong answers completely. The IR system matches a specific word very well, and it just returns that. For example, question 22 says “Nov. 28, 1929! This...sir!” It just matches “1929”, which is what BM25 does as a “naive” searcher. A fix would require time tagging and possible removal, but we did not do this as we thought it provided enough information to not be removed. Other things in this class just include tweaking BM25, but we did not create a test dataset to try to do any parameter tweaking.

Improved Implementation

Despite doing five adjustments, we only kept four of them for our improved implementation. We added stop-word removal for common Jeopardy terms, category awareness, lead paragraph upweighting, multi-word named entity upweighting, and LLM reranking. Of these, we did not keep multi-word named entity upweighting because it did not improve the score, but it could have benefits in some other corpus. We did much more than this and tried positional indexing, proximity search, syntactic dependencies, synonyms/redirects, but as mentioned in the section for measuring performance, these did not create measurable increases to our $P@1$ and often lowered the MRR of the system. One last possible improvement that could have been done was time tagging, but this tagging likely would not produce good results like how stop-words are not sufficient for a search.

Stop-word removal for common Jeopardy terms was a must as they would not be removed with a typical stop-word remover. Terms such as “this”, “these”, “who”, “whom”, “whose” are removed as they provide little contextual meaning that an IR system can parse. This increased our MRR by 0.001, but had no increase in $P@1$ (compared to the core).

Categories are also not indexed at all in the core implementation, but they can be vital as some Wikipedia pages may have matching categorical terms listed on them. This just gives more “clues” as to what could be matched. This increased our $P@1$ by 0.07, and our MRR by 0.073 (compared to the core). Some categories such as “STATE OF THE ART MUSEUM” or “CAPITAL CITY CHURCHES” could benefit from being able to match the categories appropriately.

Multi-word named entities is just a fancy way of saying proper nouns with two or more words. This could benefit some queries like “Don Knotts took over from Norman Fell as the resident landlord in this sitcom” by weighting these rare terms higher. This did not improve our scores, and it actually dropped our MRR by 0.01. It could be that some such terms should simply be understood like stopwords: “In 2001: The president of the United States on television” is one example. “United States” is mentioned in many articles, so attempting to weigh that more heavily may retrieve extraneous results with higher scores, when “president” should be given more.

Lead paragraph upweighting was a similar idea to pronoun upweighting; instead, we think that most answers, if searchable with an IR, would show up within the first paragraph of a Wikipedia article summarizing the entity. For example, question 2 talks about “this island near mainland China” which is likely to occur in the leading paragraph of the corresponding Wikipedia article for “Taiwan”. This improved our P@1 by 0.08 and MRR by 0.0528.

For LLM reranking, it produced no measurable impact as many of the error classes caused the top $K = 10$ documents to not have the correct answer. Increasing this K to 20 or even 100 still did not retrieve the correct document. However, the code is still there to provide proof of work for some of the other improvements as well. It had no measurable impact on our implementation, so we kept it.

All of these improvements resulted in getting a P@1 of 0.34 with an MRR of 0.4276.

Summary of Implementation Scores

Core Implementation:

P@1 = 0.19 (19%), MRR = 0.3150

Improved Implementation:

P@1 = 0.34 (34%), MRR = 0.4276